Week 4: Gibbs Sampling and Webscraping

30/01/26

Gibbs Sampling

In the first part of this lab we will write a Gibbs Sampler to get posterior estimates of the mean and variance of kid's test scores. Firstly, let's read in the data:

```
library(tidyverse)
kidiq <- read_rds("../data/kidiq.RDS")
```

Set up data, and choose some values for the prior hyperparameters:

```
y <- kidiq$kid_score
mean_y <- mean(y)
var_y <- var(y)
sd_y <- sd(y)
n <- length(y)

# prior settings
mu0 <- 0
nu0<-5
sigma_mu0 <-  sigma_0 <- 100
nun <- nu0+n
sigma2_mu0 <- sigma_mu0^2
sigma2_0 <- sigma_0^2
```

Do the sampling (fill in the gaps!)

```r
set.seed(123)
S <-1000
phi <- c(mean_y, 1/var_y)
phi_sp <- matrix(nrow=S,ncol=2) # matrix with dim (nsim, npar)
phi_sp[1,] <- phi
for(s in 2:S) {
  # generate a new mu value from its full conditional

  # generate a new 1/sigma^2 value from its full conditional

  phi_sp[s,] <- phi
}
```

Look at quantiles:

```r
quantile(phi_sp[,1],c(.025,.5,.975))
quantile(phi_sp[,2],c(.025,.5, .975))
quantile(1/sqrt(phi_sp[,2]),c(.025,.5, .975))
```

Plot the samples:

```r
par(lwd = 3, cex.axis = 1.5, cex.lab = 2, cex.main = 2, mfrow = c(1,3), mar=c(5,5,1,1))
for (m1 in c(5,15,100)){
  plot( phi_sp[1:m1,],type="l",xlim=range(phi_sp[1:100,1]), ylim=range(phi_sp[1:100,2]),
        lty=1,col="gray",xlab=expression(mu),ylab=expression(1/sigma^2))
  text(  phi_sp[1:m1,1], phi_sp[1:m1,2], c(1:m1), cex = 2)
}
```

Exercises

- Play around with different initial values
- Plot traceplots

Webscraping

Introduction

There's a bunch of different ways to web-scrape, but we'll be exploring using the `rvest` package in R, that helps you to deal with parsing html.

Why is web scraping useful? If our research involves getting data from a website that isn't already in a easily downloadable form, it improves the reproducibility of our research. Once you get a scraper working, it's less prone to human error than copy-pasting, for example, and much easier for someone else to see what you did.

A note on responsibility

Seven principles for web-scraping responsibly:

1. Try to use an API.
2. Check robots.txt. (e.g. https://www.utoronto.ca/robots.txt)
3. Slow down (why not only visit the website once a minute if you can just run your data collection in the background while you're doing other things?).
4. Consider the timing (if it's a retailer then why not set your script to run overnight?).
5. Only scrape once (save the data as you go and monitor where you are up to).
6. Don't republish the data you scraped (cf datasets that create based off it).
7. Take ownership (add contact details to your scripts, don't hide behind VPNs, etc)

Extracting the Billboard top 100 songs list

We're going to scrape the current Billboard top 100 songs list https://www.billboard.com/charts/hot-100/. While the data are nicely presented, there's no nice way of downloading the data for each year as a csv or similar form. So let's use `rvest` to extract the data. We'll also load in `janitor` to clean up column names etc later on.

```
library(tidyverse)
library(rvest)
library(janitor)
```

Firstly, read the html from the webpage:

```
link <- "https://www.billboard.com/charts/hot-100/"
billboard_page <- read_html(link)
```

We want to extract information from the (interactive) table. To find where this was in the html, I made use of the 'Inspect' option when you right-click in Chrome. This was a relatively straightforward way of find the right class for each table item. Once I had that, I pulled all the html nodes of that class, converted to text, removed all "\n" (tabs), converted to a tibble and then separated the values. The final line of code gets rid of all empty columns.

```
# class of table objects
class <- "o-chart-results-list-row-container"

rough_table <- billboard_page |>
  html_nodes(xpath = paste0("//div[@class = '",class,"']"))|> # pull out table
  html_text() |> # convert to text
  str_remove_all("\t") |> # remove tabs
  as_tibble() |> # convert to tibble
  separate(value, into = paste0(1:1000), sep = "\n") |> # separate into columns
  select_if(function(x) !(all(is.na(x)) | all(x==""))  # remove empty columns
```

We're almost there, but some of the values are mis-aligned, because some songs have an additional label of 'new' or a 're-entry'. To tidy this up, I found it easiest to separate these songs out, clean up the table, and then join them back together.

```
# new songs tidying up
rough_new <- rough_table |>
  filter(`12`=="NEW")|>
  select_if(function(x) !(all(is.na(x)) | all(x==""))) |>
  rename(position = `5`,
         song_name = `43`,
         artist = `48`,
         peak_position = `79`,
         weeks_in_chart = `92`
         ) |>
    mutate(last_week = "-") |>
  select(position, song_name, artist, last_week, peak_position, weeks_in_chart) |>
  mutate(label = "new")

# re-entry songs tidying up
rough_re <- rough_table |>
  filter(`12` == "RE-") |>
  mutate(`49` = ifelse(`49`=="", `50`, `49`),
         `80` = ifelse(`80`=="", `82`, `80`),
         `93` = ifelse(`93`=="", `95`, `93`))|>
  rename(position = `5`,
         song_name = `44`,
         artist = `49`,
         peak_position = `80`,
         weeks_in_chart = `93`
         ) |>
  mutate(last_week = "-") |>
```

```
  select(position, song_name, artist, last_week, peak_position, weeks_in_chart) |>
  mutate(label = "re-entry")

# everything else tidying up
rough_everything_else <- rough_table |>
  filter(`12` == "") |>
  mutate(`44` = ifelse(`44`=="", `45`, `44`),
         `60` = ifelse(`60`=="", `62`, `60`),
          `86` = ifelse(`86`=="", `88`, `86`),
          `73` = ifelse(`73`=="", `75`, `73`))|>
  rename(position = `5`,
         song_name = `39`,
         artist = `44`,
         last_week = `60`,
         peak_position = `73`,
         weeks_in_chart = `86`
         ) |>
  select(position, song_name, artist, last_week, peak_position, weeks_in_chart) |>
  mutate(label = "NA")

# bind them all together and sort
clean_table <- bind_rows(rough_everything_else,
                         rough_new, rough_re) |>
  mutate(position = as.numeric(position)) |>
  arrange(position)

clean_table
```

```
# A tibble: 100 x 7
   position song_name        artist last_week peak_position weeks_in_chart label
      <dbl> <chr>            <chr>  <chr>     <chr>         <chr>          <chr>
 1        1 I Just Might     Bruno~ "1"       "1"           "2"            NA
 2        2 Man I Need       Olivi~ "4"       "2"           "22"           NA
 3        3 The Fate Of Oph~ Taylo~ "2"       "1"           "16"           NA
 4        4 Golden           HUNTR~ "3"       "1"           "31"           NA
 5        5 Ordinary         Alex ~ "5"       "1"           "49"           NA
 6        6 Choosin' Texas   Ella ~ ""        ""            ""             NA
 7        7 Back To Friends  sombr  "8"       "7"           "43"           NA
 8        8 Folded           Kehla~ "9"       "6"           "32"           NA
 9        9 End Of Beginning Djo    "7"       "6"           "25"           NA
10       10 Opalite          Taylo~ "10"      "2"           "16"           NA
# i 90 more rows
```

Install rstan and brms

We will be using the packages `rstan` and `brms` from next week. Please install these. Here's some instructions:

- https://github.com/paul-buerkner/brms
- https://github.com/stan-dev/rstan/wiki/RStan-Getting-Started

In most cases it will be straightforward and may not need much more than `install.packages()`, but you might run into issues. Every Stan update seems to cause problems for different OS.

To make sure it works, run the following code:

```
library(brms)

x <- rnorm(100)
y <- 1 + 2*x + rnorm(100)
d <- tibble(x = x, y= y)

mod <- brm(y~x, data = d)
```

```
Running /Library/Frameworks/R.framework/Resources/bin/R CMD SHLIB foo.c
using C compiler: 'Apple clang version 12.0.0 (clang-1200.0.31.1)'
using SDK: 'MacOSX11.0.sdk'
clang -arch x86_64 -I"/Library/Frameworks/R.framework/Resources/include" -
DNDEBUG   -I"/Library/Frameworks/R.framework/Versions/4.5-x86_64/Resources/library/Rcpp/inclu
I"/Library/Frameworks/R.framework/Versions/4.5-x86_64/Resources/library/RcppEigen/include/"
I"/Library/Frameworks/R.framework/Versions/4.5-x86_64/Resources/library/RcppEigen/include/uns
I"/Library/Frameworks/R.framework/Versions/4.5-x86_64/Resources/library/BH/include" -
I"/Library/Frameworks/R.framework/Versions/4.5-x86_64/Resources/library/StanHeaders/include/s
I"/Library/Frameworks/R.framework/Versions/4.5-x86_64/Resources/library/StanHeaders/include/"
I"/Library/Frameworks/R.framework/Versions/4.5-x86_64/Resources/library/RcppParallel/include/
I"/Library/Frameworks/R.framework/Versions/4.5-x86_64/Resources/library/rstan/include" -
DEIGEN_NO_DEBUG  -DBOOST_DISABLE_ASSERTS  -DBOOST_PENDING_INTEGER_LOG2_HPP  -
DSTAN_THREADS  -DUSE_STANC3 -DSTRICT_R_HEADERS  -DBOOST_PHOENIX_NO_VARIADIC_EXPRESSION  -
D_HAS_AUTO_PTR_ETC=0  -include '/Library/Frameworks/R.framework/Versions/4.5-
x86_64/Resources/library/StanHeaders/include/stan/math/prim/fun/Eigen.hpp'  -
D_REENTRANT -DRCPP_PARALLEL_USE_TBB=1   -I/opt/R/x86_64/include    -fPIC  -
falign-functions=64 -Wall -g -O2  -c foo.c -o foo.o
In file included from <built-in>:1:
In file included from /Library/Frameworks/R.framework/Versions/4.5-x86_64/Resources/library/S
In file included from /Library/Frameworks/R.framework/Versions/4.5-x86_64/Resources/library/R
In file included from /Library/Frameworks/R.framework/Versions/4.5-x86_64/Resources/library/R
```

```
/Library/Frameworks/R.framework/Versions/4.5-x86_64/Resources/library/RcppEigen/include/Eige
#include <cmath>
         ^~~~~~~
1 error generated.
make: *** [foo.o] Error 1


SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
Chain 1:
Chain 1: Gradient evaluation took 2.6e-05 seconds
Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 0.26 seconds.
Chain 1: Adjust your expectations accordingly!
Chain 1:
Chain 1:
Chain 1: Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 1: Iteration:  200 / 2000 [ 10%]  (Warmup)
Chain 1: Iteration:  400 / 2000 [ 20%]  (Warmup)
Chain 1: Iteration:  600 / 2000 [ 30%]  (Warmup)
Chain 1: Iteration:  800 / 2000 [ 40%]  (Warmup)
Chain 1: Iteration: 1000 / 2000 [ 50%]  (Warmup)
Chain 1: Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 1: Iteration: 1200 / 2000 [ 60%]  (Sampling)
Chain 1: Iteration: 1400 / 2000 [ 70%]  (Sampling)
Chain 1: Iteration: 1600 / 2000 [ 80%]  (Sampling)
Chain 1: Iteration: 1800 / 2000 [ 90%]  (Sampling)
Chain 1: Iteration: 2000 / 2000 [100%]  (Sampling)
Chain 1:
Chain 1:  Elapsed Time: 0.021 seconds (Warm-up)
Chain 1:                0.021 seconds (Sampling)
Chain 1:                0.042 seconds (Total)
Chain 1:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 2).
Chain 2:
Chain 2: Gradient evaluation took 5e-06 seconds
Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 0.05 seconds.
Chain 2: Adjust your expectations accordingly!
Chain 2:
Chain 2:
Chain 2: Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 2: Iteration:  200 / 2000 [ 10%]  (Warmup)
Chain 2: Iteration:  400 / 2000 [ 20%]  (Warmup)
```

```
Chain 2: Iteration:  600 / 2000 [ 30%]  (Warmup)
Chain 2: Iteration:  800 / 2000 [ 40%]  (Warmup)
Chain 2: Iteration: 1000 / 2000 [ 50%]  (Warmup)
Chain 2: Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 2: Iteration: 1200 / 2000 [ 60%]  (Sampling)
Chain 2: Iteration: 1400 / 2000 [ 70%]  (Sampling)
Chain 2: Iteration: 1600 / 2000 [ 80%]  (Sampling)
Chain 2: Iteration: 1800 / 2000 [ 90%]  (Sampling)
Chain 2: Iteration: 2000 / 2000 [100%]  (Sampling)
Chain 2:
Chain 2:  Elapsed Time: 0.022 seconds (Warm-up)
Chain 2:                0.018 seconds (Sampling)
Chain 2:                0.04 seconds (Total)
Chain 2:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 3).
Chain 3:
Chain 3: Gradient evaluation took 6e-06 seconds
Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 0.06 seconds.
Chain 3: Adjust your expectations accordingly!
Chain 3:
Chain 3:
Chain 3: Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 3: Iteration:  200 / 2000 [ 10%]  (Warmup)
Chain 3: Iteration:  400 / 2000 [ 20%]  (Warmup)
Chain 3: Iteration:  600 / 2000 [ 30%]  (Warmup)
Chain 3: Iteration:  800 / 2000 [ 40%]  (Warmup)
Chain 3: Iteration: 1000 / 2000 [ 50%]  (Warmup)
Chain 3: Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 3: Iteration: 1200 / 2000 [ 60%]  (Sampling)
Chain 3: Iteration: 1400 / 2000 [ 70%]  (Sampling)
Chain 3: Iteration: 1600 / 2000 [ 80%]  (Sampling)
Chain 3: Iteration: 1800 / 2000 [ 90%]  (Sampling)
Chain 3: Iteration: 2000 / 2000 [100%]  (Sampling)
Chain 3:
Chain 3:  Elapsed Time: 0.024 seconds (Warm-up)
Chain 3:                0.022 seconds (Sampling)
Chain 3:                0.046 seconds (Total)
Chain 3:

SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 4).
Chain 4:
Chain 4: Gradient evaluation took 5e-06 seconds
```

```
Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 0.05 seconds.
Chain 4: Adjust your expectations accordingly!
Chain 4:
Chain 4:
Chain 4: Iteration:    1 / 2000 [  0%]  (Warmup)
Chain 4: Iteration:  200 / 2000 [ 10%]  (Warmup)
Chain 4: Iteration:  400 / 2000 [ 20%]  (Warmup)
Chain 4: Iteration:  600 / 2000 [ 30%]  (Warmup)
Chain 4: Iteration:  800 / 2000 [ 40%]  (Warmup)
Chain 4: Iteration: 1000 / 2000 [ 50%]  (Warmup)
Chain 4: Iteration: 1001 / 2000 [ 50%]  (Sampling)
Chain 4: Iteration: 1200 / 2000 [ 60%]  (Sampling)
Chain 4: Iteration: 1400 / 2000 [ 70%]  (Sampling)
Chain 4: Iteration: 1600 / 2000 [ 80%]  (Sampling)
Chain 4: Iteration: 1800 / 2000 [ 90%]  (Sampling)
Chain 4: Iteration: 2000 / 2000 [100%]  (Sampling)
Chain 4:
Chain 4:  Elapsed Time: 0.022 seconds (Warm-up)
Chain 4:                0.02 seconds (Sampling)
Chain 4:                0.042 seconds (Total)
Chain 4:
```

```
summary(mod)
```

```
 Family: gaussian
  Links: mu = identity
Formula: y ~ x
   Data: d (Number of observations: 100)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

Regression Coefficients:
          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept     1.12      0.10     0.92     1.33 1.00     3074     2603
x             1.97      0.09     1.81     2.14 1.00     3791     2641

Further Distributional Parameters:
      Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
sigma     1.02      0.07     0.89     1.17 1.00     4559     3006

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
```

```
scale reduction factor on split chains (at convergence, Rhat = 1).
```